Тестовое задание для Backend-разработчика (Java)

Важно: ссылку на выполненное тестовое необходимо указать в резюме в блок «О себе».

Срок выполнения: 3 дня.

Формат сдачи: ссылка на репозиторий.

Бизнес-кейс: Контроль качества данных по целям сотрудников

Проект: реализовать требования бизнес-заказчика в части расчёта и визуализации процедуры контроля качества данных (далее — ККД). В корпоративной системе управления целями появится модуль ККД: в нём администраторы системы, руководители департаментов и ответственные за целеполагание в бизнес-подразделениях на периодической основе (планируется еженедельно) проверяют качество данных по целям и ключевым результатам.

Проблема: сотрудники заносят цели и ключевые результаты в корпоративную систему управления целями, и качество этих данных нужно отслеживать. Если цели или ключевые результаты заведены некорректно, процесс целеполагания работает неэффективно, а руководство компании не видит реальной картины по работе над целями, которые само же и ставит.

Как процесс работает сейчас (AS IS)

Процесс ККД сейчас считает технические метрики, без нужной визуализации.

Как процесс должен работать (TO BE)

- Механизм ККД работает по существующим данным целей и ключевых результатов в PostgreSQL.
- Подсчитываемые метрики соответствуют бизнес-требованиям и учитывают каскадирование целей.
- Endpoint-ы в существующих сервисах на Java передают результаты работы модуля ККД на фронтенд.
- В системе управления целями реализована страница модуля ККД: статистика, найденные проблемы и дельта с прошлым прогоном по данным.
- Руководители и администраторы системы проверяют качество данных на периодической основе — планируется еженедельно.

Ключевые сценарии использования (User Stories)

- Как пользователь сервиса, я хочу видеть количественные и качественные метрики по существующему набору данных (цели и ключевые результаты) с учётом каскадирования.
- Как пользователь сервиса, я хочу видеть динамику (дельту) по сравнению с предыдущим набором данных, полученным в прошлый раз.
- Как разработчик фронтенда, я хочу получать через API метрики, найденные проблемы и дельту с прошлым прогоном, чтобы показать их на странице ККД без расчётов на клиенте.

Тестовое задание

Задача: реализовать сервис модуля ККД на Java: проверку качества данных по целям и ключевым результатам, хранение результатов в PostgreSQL и REST API для страницы ККД.

- Спроектировать схему БД: подразделения с иерархией, цели (ответственный, родительская цель, статус, срок) и ключевые результаты (целевое и фактическое значение, вес, дата обновления факта); плюс таблицы результатов ККД: справочник правил, прогон и найденная проблема.

— Реализовать 4 правила на выбор из списка ниже. Логику правил держать отдельно от слоя API и доступа к данным — отдельным компонентом, который получает данные и возвращает список найденных проблем.

— Загрузить стартовые данные: 20–30 целей в двух-трёх подразделениях и два прогона — так, чтобы были нарушения каждого правила и было с чем сравнивать.

— Миграции — Liquibase или Flyway с откатом: накат → откат → повторный накат без ошибок (в проекте это условие приёмки на общем тестовом стенде). Весь проект поднимается одной командой docker compose up — сервис, PostgreSQL, миграции и стартовые данные.

— Реализовать GET /api/runs — список прогонов: статус, дата снимка, с каким прогоном сравнивается, число найденных проблем.

— Реализовать GET /api/kkd/summary?run_id=&unit_id= — минимум три метрики по подразделению с текущим и прошлым значением, дельтой и признаком «лучше / хуже», разбивка проблем по важности и счётчики новых, сохраняющихся и исправленных. Метрики подразделения считаются с учётом дочерних, формат ответа — ниже.

— Реализовать GET /api/kkd/issues — список проблем прогона с фильтрами по подразделению, правилу и важности и пагинацией (limit, offset, total).

— Дельта: проблема «та же самая», что и в прошлом прогоне, если совпадают правило и объект. У первого прогона сравнения нет — прошлые значения и дельта равны null, а не 0.

— Проверки: неизвестный прогон или подразделение — 404, некорректные параметры — 422. Формат ошибки единый: {"error": {"code": "...", "message": "..."}}.

— Покрыть тестами (JUnit 5) минимум 4 сценария: два правила на граничных данных, расчёт метрики и расчёт дельты. Поднимать HTTP не требуется.

— **Если решение написано с ИИ-ассистентом** — в репозитории должен быть один артефакт на выбор. Либо папка specs/ со спецификацией, по которой шла работа: requirements.md (что делает решение и критерии приёмки) и design.md (модель данных, API, ключевые решения); если решение менялось по ходу — спецификация обновлена. Либо файл AI-PLAN.md — план работы с ассистентом: какие задачи ему ставились, что принято как есть, что переписано вручную и почему.

— **Если решение написано без ИИ-ассистента** — ничего дополнительно прикладывать не нужно, достаточно одной строки об этом в README.

Правила ККД — выбери любые 4

— GOAL_NO_OWNER — цель без ответственного (critical).

— GOAL_NO_KR — у активной цели нет ни одного ключевого результата (critical).

— KR_WEIGHTS_SUM — сумма весов КР цели не равна 100% (major).

— CASCADE_NO_PARENT — цель не связана с целью вышестоящего уровня (major).

— KR_NO_TARGET — у КР не заполнено целевое значение (major).

— KR_STALE_PROGRESS — факт КР не обновлялся более 45 дней (minor).

Уровни важности: critical, major, minor. Правила применяются только к активным целям и их КР.

Формат ответа GET /api/kkd/summary (фрагмент)

```
{
  "run": { "id": "RUN-008", "snapshot_date": "2026-10-26" },
  "compared_to": { "id": "RUN-007", "snapshot_date": "2026-10-19" },
```

```
  "unit": { "id": "HQ-CIB", "name": "Департамент корпоративного бизнеса" },
  "metrics": [
    { "key": "clean_goals_share", "title": "Доля корректных целей", "unit":
"%",
      "value": 56.5, "previous": 62.7, "delta": -6.2, "trend": "worse" }
  ],
  "issues_by_severity": [
    { "code": "critical", "current": 2, "previous": 2, "delta": 0 },
    { "code": "major", "current": 23, "previous": 15, "delta": 8 }
  ],
  "delta": { "new": 10, "persisting": 33, "resolved": 2 }
}
```

Технические требования

Стек: Java 17+, Spring Boot 3, PostgreSQL 15+, Liquibase или Flyway, Maven.

Конфигурация через переменные окружения (.env.example), корректные HTTP-статусы, валидация входных параметров, контроллеры отделены от сервисного слоя и доступа к данным.

ИИ-ассистент (GigaCode / GigaIDE или любой другой) разрешён и на оценку не влияет — важен результат и артефакт из пункта выше.

Что должно быть в README

- Как запустить проект, тесты и проверить накат → откат → повторный накат миграций.
- Архитектура решения: где живёт логика правил, как считаются метрики и дельта между прогонами.
- Писалось решение с ИИ-ассистентом или без. Если с ассистентом — ссылка на specs/ или AI-PLAN.md.

Если приложена спецификация или план работы с ассистентом, отдельно оценивается, соответствует ли им итоговый код.

Мотивация участия в проекте

- Почему тебе интересен этот проект?
- Как ты видишь свою роль в команде, которая создаёт продукт?
- Сколько времени в неделю ты готов(а) уделять проекту и в течение какого периода?